EXPERIMENT 4: Morphological Analysis using Finite State Automata (FSA)

AIM:

To implement morphological analysis using a Finite State Automata (FSA) model with defined states (START, PREFIX, ROOT, SUFFIX, ACCEPT) for more structured word segmentation.

PROCEDURE:

1. Import re module.
2. Define sample text and preprocess it.
3. Define prefix list and suffix list.
4. Define FSA states: START=0, PREFIX=1, ROOT=2, SUFFIX=3, ACCEPT=4.
5. Define fsa_morphological_analyzer(word):

- Initialize state = START, prefix, root, suffix.
- PREFIX STATE: Check if word starts with any prefix; if yes, set prefix, root = word[len(p):], state = PREFIX. - If state == START, set state = ROOT.
- SUFFIX STATE: Check if root ends with any suffix; update root and suffix, state = SUFFIX.
- ACCEPT STATE: Set state = ACCEPT.
- Return prefix, root, suffix.

6. For each token, call fsa_morphological_analyzer and print results.

CODE:

```
import re


text = """
The players replayed matches happily.
Workers are working tirelessly and restarted machines.
"""


def preprocess(text):    text =
text.lower()    text = re.sub(r'[^a-
z\s]', '', text)    tokens =
text.split()    return tokens


tokens = preprocess(text)


prefixes = ["un", "re", "pre", "dis", "mis"] suffixes
= ["ing", "ed", "ly", "er", "s"]
START = 0
PREFIX = 1
ROOT = 2
SUFFIX = 3
ACCEPT = 4


def fsa_morphological_analyzer(word):
state = START    prefix = ""    root
= word    suffix = ""


    # ---- PREFIX STATE ----
for p in prefixes:        if
word.startswith(p):
prefix = p            root =
word[len(p):]            state
= PREFIX            break


    if state == START:
state = ROOT


    # ---- SUFFIX STATE ----
for s in suffixes:
if root.endswith(s):
            suffix = s
root = root[:-len(s)]
state = SUFFIX            break


    # ---- ACCEPT STATE ----
state = ACCEPT


    return prefix, root, suffix
```

```
print("Finite State Automata Morphological Analysis\n")
for word in tokens:     p, r, s =
fsa_morphological_analyzer(word)     print(f"Word:
{word}")
    print(f"Prefix: {p if p else 'None'}")
print(f"Root: {r}")
    print(f"Suffix: {s if s else 'None'}")
print("----------------------")
```

OUTPUT:

```
Finite State Automata Morphological Analysis


Word: the
Prefix: None  Root: the  Suffix: None
Word: players
Prefix: None  Root: player  Suffix: s Word:
replayed
Prefix: re  Root: play  Suffix: ed Word:
matches
Prefix: None  Root: matche  Suffix: s
Word: happily
Prefix: None  Root: happi  Suffix: ly
Word: workers
Prefix: None  Root: worker  Suffix: s
```

```
Word: are
Prefix: None  Root: are  Suffix: None
Word: working
Prefix: None  Root: work  Suffix: ing
Word: tirelessly
Prefix: None  Root: tireless  Suffix: ly
Word: restarted
Prefix: re  Root: start  Suffix: ed
Word: machines
Prefix: None  Root: machine  Suffix: s
```

RESULT:

The FSA-based morphological analyzer was successfully implemented with clearly defined states (START, PREFIX, ROOT, SUFFIX, ACCEPT). The FSA model transitions through states systematically, correctly segmenting words into prefix, root, and suffix components. Results are consistent with the rule-based approach, validating the FSA implementation.